

Agent Teams vs. subagentes: como investigar um bug intermitente sem estourar o orçamento

Resumo da pesquisa feita após a prática "O Especialista em Orquestração Multi-Agente e o Gerente com um Bug Impossível" — quando vale a pena trocar subagentes isolados por um Agent Team colaborativo, e como fazer isso sem corromper o código nem explodir o consumo de tokens.

Fernando Melo · 9 de setembro de 2026

01 · O PROBLEMA

Um bug intermitente, um prazo curto e uma dúvida sobre "Agent Teams"

O app fecha sozinho às vezes, sem padrão aparente, e ninguém da equipe conseguiu isolar a causa. O time já usa subagentes para pesquisar, mas ouviu falar de "Agent Teams" e ficou com duas dúvidas: isso não seria a mesma coisa que subagentes com outro nome? E, se vários agentes forem soltos ao mesmo tempo sobre o código, eles não vão pisar uns nos outros — bagunçando o repositório e explodindo a conta de tokens?

A resposta curta: não são a mesma coisa, a diferença está na arquitetura de comunicação, e ativar um time sem controle pode sim corromper o código por conflitos de escrita, além de multiplicar o consumo de tokens.

02 · SUBAGENTES VS. AGENT TEAMS

Hub-and-spoke contra esquadrão em tempo real

Subagentes seguem um modelo centralizado: cada um executa uma tarefa isolada e reporta ao agente principal. Um Agent Team opera como um esquadrão em tempo real, no qual as instâncias conversam lateralmente entre si, debatem e atualizam uma lista de tarefas compartilhada.

CARACTERÍSTICA	SUBAGENTES	AGENT TEAMS
Comunicação	Apenas vertical (com o agente principal)	Lateral e direta entre os membros do time
Consumo de tokens	Mais econômico — o histórico morre após a entrega	Altíssimo — mensagens cruzadas inflam o contexto de todos
Risco de conflito	Baixo — atuação focada e sequencial	Alto — agentes paralelos podem assumir premissas conflitantes
Ideal para	Investigação de logs, buscas e tarefas independentes	Projetos complexos que tocam várias camadas (ex.: front + back)

Isolamento que nunca conversa vs. colaboração em tempo real

Subagentes (isolamento): o agente principal cria o Subagente A para ler o arquivo de rede; ele termina, entrega o relatório e deixa de existir. Depois, o principal cria o Subagente B para ler os logs. A e B nunca conversaram entre si.

Agent Teams (colaboração): os agentes coexistem em tempo real. O Agente de Logs encontra um comportamento estranho no banco de dados e avisa diretamente o Agente de Código, que responde "isso explica por que a thread X trava" — resolvendo o quebra-cabeça juntos, sem passar pelo agente principal a cada troca.

Só um agente escreve no disco — os demais só debatem

Para usar um time sem estourar o orçamento nem estragar o repositório, a divisão de papéis segue uma regra simples: apenas um agente tem permissão de escrita.

Agente Líder — único com permissão Git

Controla o fluxo e os custos, recebe o comando do usuário e decide os próximos passos do time.

Agente de Logs (apenas leitura)

Analisa crash dumps, stack traces e comportamento de memória.

Agente de Código (apenas leitura)

Analisa condições de corrida, concorrência e o fluxo do código-fonte.

Logs e Código debatem entre si em tempo real, mas nenhum dos dois altera arquivos diretamente — só o Agente Líder aplica a correção, e apenas depois do aval do desenvolvedor.

- 1 Separação estrita de papéis:** três agentes no time, mas escrita em disco liberada para apenas um deles.
- 2 Fluxo de duas fases:** Fase 1 (diagnóstico) — o time debate as causas e apresenta um relatório consolidado, com qualquer alteração de código bloqueada. Fase 2 (correção isolada) — só depois do diagnóstico correto, o time é desativado e o Agente Diretor aplica a correção pontual numa branch Git isolada.
- 3 Travas de segurança financeira:** limite de 5 a 7 rodadas de conversa (max turns) e contexto enxuto — nunca o repositório inteiro, só as pastas suspeitas e os logs pertinentes.

De 1x a quase 7x o custo de uma rodada com subagentes

O custo escala por causa do "efeito broadcast": cada mensagem gerada por um agente é replicada no histórico de todos os outros membros do time na iteração seguinte. Um subagente que termina descarta seu histórico; num Agent Team, o histórico acumula e infla o faturamento a cada nova réplica.

ESTRUTURA	INPUT TOTAL	OUTPUT TOTAL	CUSTO ESTIMADO	MULTIPLICADOR
Subagentes	~66.000 tokens	~4.000 tokens	~US\$ 0,25	1x (base)
Team (3 agentes)	~315.000 tokens	~15.000 tokens	~US\$ 1,17	4,7x mais caro
Team (4 agentes)	~460.000 tokens	~20.000 tokens	~US\$ 1,68	6,9x mais caro

Atenção aos custos: um único debate que dure 20 ou 30 rodadas com 4 agentes pode facilmente queimar entre US\$ 10,00 e US\$ 20,00 numa única janela de depuração, se o contexto do repositório for muito amplo.

Subagentes baratos limpam o terreno; o time caro só bate o martelo

Em vez de deixar um time de 4 agentes rodando o tempo todo, a recomendação para um prazo curto é um modelo "funil": subagentes baratos primeiro, Agent Team cirúrgico só no final.

- Fase 1 (subagente, barato):** um subagente isolado lê os logs brutos e extrai as ~30 linhas relevantes do stack trace, resumindo tudo em texto puro.
- Fase 2 (subagente, barato):** outro subagente mapeia quais arquivos controlam a lógica do erro.
- Fase 3 (Agent Team, cirúrgico):** um time enxuto de 2 agentes (ex.: Especialista em Concorrência + Especialista em Memória) recebe apenas os resumos das fases 1 e 2 — não o repositório e os logs gigantes — e debate sobre dados já mastigados, o que reduz drasticamente o custo inicial.

Três ajustes para cortar o desperdício de tokens no time

Padrão em estrela (star topology): por padrão os times usam broadcast — todo mundo fala com todo mundo. Configurar o framework (CrewAI, LangGraph) para o modo hierárquico faz os especialistas conversarem só com um "gerente", cortando as mensagens cruzadas pela metade.

Modelos assimétricos: nem todo agente do time precisa do modelo mais caro do mercado. O agente que só lê e busca padrões no log pode usar um modelo mais rápido e barato; o modelo mais caro e inteligente fica reservado para o Agente Líder, que toma a decisão final de arquitetura.

Prompt de resumo de turno (context compression): uma instrução sistêmica exigindo que cada agente seja extremamente sucinto.

"Responda em no máximo 3 tópicos técnicos. Não repita o que o agente anterior disse. Vá direto à sua discordância ou validação."

Transparência sobre custo e risco evita surpresa na fatura

Agir com transparência total sobre custos, riscos e limitações evita surpresas na fatura e impede que o código seja corrompido às vésperas do prazo. Resumo operacional para levar à equipe:

Status tecnológico: recurso experimental (em preview), muitas vezes exigindo ativação manual via flags de configuração — mais estável quando executado direto no terminal, longe de interfaces visuais pesadas.

Impacto financeiro real: uma sessão com um time de 3 agentes consome, em média, de 2,5x a 4x mais tokens do que uma sessão com um único agente focado, devido ao tráfego de mensagens cruzadas e à replicação de contexto.

Risco de concorrência (race conditions): múltiplos agentes editando o código ao mesmo tempo geram conflitos de escrita. A regra de ouro é isolar escopos e diretórios distintos para cada agente, garantindo que suas áreas de atuação não se sobreponham no disco.